SOFTSEC VT 2026

Phase II Operational Security

Threat Modeling, Logging, Monitoring & Incident Analysis

Group 22 — Tatou PDF-Watermarking Deployment
Report date: 14 October 2026

Written By: Sopnil Roy Niloy

Executive Summary

In this paper, observation points for operational security are created for the Group 22 Tatou deployment. The changes made create authentication failure events, start Gunicorn HTTP access logging, and delete the flag value from the startup message. The invalid token request received HTTP 401 response and was recorded as an authentication failure. The result of the health check is HTTP 200, and the database is connected.

The test suite for the server was re-run on 14 October 2026 on Python 3.13.5, giving 26 passes, 7 skipped tests, and no failures. The report consists of a threat model, recovery runbook, summary of the evidence, and an excerpt from the redacted log. The recovery test verifies the deployment of the server and the operation of health checks; however, backup restore and recovery after the successful breach have not been verified yet.

1. Scope and System Overview

The Tatou system is an online application designed to watermark PDF files. The README of the project states that it is a learning project with potential vulnerabilities and suggests not to deploy it in an open network. Group 22 deploys it with the use of a Python/Flask server running in Gunicorn, MariaDB container, and Docker Compose.

In the Docker Compose file, the server runs on host port 5000, uses a separate database service, keeps documents in a named user-files volume, stores the MariaDB data in the db-data volume, and has access to a GPG folder on the host computer. Environment variables provide the configuration and secrets.

This report will address the Operational Security specialization and concentrate on the following topics: threat modeling, observation points, logging, service recovery, evidence collection, and a failed authentication attempt.

2. Threat Model

2.1 Assets and security objectives

Asset	Security objective	Potential impact
Uploaded and watermarked PDF documents	Confidentiality and integrity	Disclosure, unauthorized change, or deletion
Authentication tokens and cryptographic material	Confidentiality and integrity	Unauthorized API access or misuse of protected operations
Environment variables and application secrets	Confidentiality	Impersonation or other compromise
Database records and credentials	Confidentiality, integrity, availability	Data exposure, tampering, or service disruption
GPG key material	Confidentiality and integrity	Misuse of cryptographic operations
Application and	Confidentiality, integrity,	Evidence loss, misleading investigations, or

security logs	availability	sensitive-data disclosure
Server and database availability	Availability	Interruption of API and watermarking operations

2.2 Threat actors and assumptions

Threat actors can include unauthorized users accessing API endpoints, users offering unacceptable or invalid tokens, authorized users executing unauthorized document actions, or any actors/activities accessing the host, container, mounted files, or deployment configuration itself. This is considered a threat scenario and not an assumption that the relevant threat actor has compromised the deployment itself.

2.3 Threat scenarios, observation points, and response

ID	Scenario	Observation point	Mitigation / response
T1	Repeated missing/invalid-token requests	Authentication-failure events and HTTP access logs	Review event frequency and restrict access when justified and authorized
T2	Expired token presented to protected endpoint	Authentication event with reason expired_token	Reject request and investigate repeated failures; live validation remains outstanding
T3	Secret disclosure through logs	Startup, application, and container logs	Avoid logging secret values; restrict log access and redact evidence
T4	Hostile document requests or uploads	Access and relevant application error logs	Preserve evidence, inspect request context, and investigate affected documents
T5	Unauthorized/destructive document operations	Application events and document/database state	Assess impact and restore from verified backups if available
T6	Container compromise or excessive privilege	Container configuration and runtime inspection	Assess least privilege, capabilities, and filesystem permissions
T7	Service disruption	Access logs, container status, health endpoint, error logs	Preserve logs, restrict traffic if appropriate, recover service, and verify health

3. Implemented Changes

3.1 Authentication-failure logging

The guard used to authenticate users in server/src/server.py logs security-related failure events when there are issues with authorization header, expired token, or invalid tokens. In each log message, the reason behind the failure, target endpoint, HTTP method used, and remote address are included. However, token values are not included in these log messages in order to preserve security.

Coverage of test case in server/test/test_api.py examines logging of security failure related to absence of authorization and invalid tokens. There is a branch in the code that logs expired tokens; however, this was not demonstrated in the live deployment during validation.

3.2 HTTP access logging

In server/entrypoint.sh, Gunicorn was configured with access and error logs directed to standard output. The observed access records include the request path, HTTP method, status code, and user-agent information.

3.3 Safer startup message

The startup message was changed so it reports that a configured value is used without printing the configured flag value. This reduces future disclosure through that message. It does not erase historical container log entries.

4. Validation and Test Results

Check	Observed result	Interpretation
Shell syntax: bash -n server/entrypoint.sh	Entrypoint syntax OK	The shell script parsed successfully
Build and redeploy	Image built; server container started	Server service was recreated without intentionally restarting the database
GET /healthz	HTTP 200; db_connected=true	Application responded and reported database connectivity
Controlled invalid-token request	GET /api/list-documents returned HTTP 401	Authentication rejected the deliberately invalid token
Security-event log	authentication_failed; reason=invalid_token	Application logged the failure without including the token
Gunicorn access log	Request recorded with HTTP 401	HTTP access logging was observed
pytest -v	26 passed, 7 skipped, 0 failed	Existing suite passed with seven skipped tests
git diff --check	No output / no whitespace errors	Diff whitespace check passed

The test run used Python 3.13.5, pytest 9.1.1, and collected 32 items with one item skipped during collection. The final summary was 26 passed and 7 skipped in 0.50 seconds. Skipped tests mean that not every test path was executed.

5. Evidence Collection

The repository contains a narrative evidence record at docs/evidence/operational-security-events.md and a redacted log extract at docs/evidence/container-log-extract-redacted.txt. The extract was produced from recent Docker Compose server logs by selecting health-check, authentication-failure, and protected-endpoint access-log lines.

Observed time (UTC)	Event	Evidence	Assessment
2026-10-14 18:30:54	Health check	GET /healthz, HTTP 200; response reported db_connected=true	Service-health check succeeded
2026-10-14 18:32:07	Controlled invalid-token test	GET /api/list-documents, HTTP 401	Unsuccessful authentication attempt
2026-10-14 18:32:07	Security event	authentication_failed; reason=invalid_token	Failure was detected and recorded

The extract is a filtered, redacted selection of container log lines, not a complete unmodified log export. The record intentionally omits the supplied test token and any configured secret values. Preserve original logs securely if formal forensic evidence is required.

6. Incident Analysis and Response

6.1 Event summary

At 18:32:07 UTC on 14 October 2026, a controlled request using a deliberately invalid test token was sent to GET /api/list-documents. The API returned HTTP 401 Unauthorized. The application logged an authentication-failure event with reason invalid_token and recorded endpoint, method, and remote address. Gunicorn also recorded the request and its HTTP status.

6.2 Response and interpretation

The response was to inspect the returned status and corresponding logs, verify that the security event was recorded, confirm that the token itself was not present in the event, and preserve the relevant fields in the evidence record. The observed request was rejected. This was a controlled test of an unsuccessful authentication attempt, not evidence of a successful compromise or of an attack by an external actor.

The logged remote address is evidence of the network peer observed by the server for this request; it does not establish the identity or intent of a person.

7. Recovery Procedure

This operational runbook can be found at docs/operational-security-recovery.md. It focuses on log preservation, checking of the services state, validation of source changes and tests, creation of the server image, re-creation of the server service only, and evaluation of service health and connectivity to the database.

The exercise of service redeployment was held on 14 October 2026. As part of this exercise, a server image has been re-created and a server container was re-created using the command docker compose up -d --no-deps server. As a result, the endpoint health returned HTTP 200 and db_connected=true.

According to the Compose file, there are volumes created for user files, which are named user-files, and also db-data for MariaDB, along with GPG directory mounted to the host. Standard recovery procedure should preserve these volumes. This procedure does not restore deleted documents and database data. No backup restoration or recovery from compromise has been tested.

8. Conclusion

The work added authentication-failure logging, enabled Gunicorn HTTP access logging, and changed the startup message to avoid printing a configured value. The deployment was validated by a successful health check and a controlled invalid-token request that was rejected and logged. The test suite reported 26 passed and 7 skipped.

The threat model, recovery runbook, evidence summary, and redacted log extract document the process and its observed results. Service redeployment was validated in the tested scenario, but restoration from backups, recovery of lost data, and response to a successful compromise remain unverified. These limitations are stated explicitly rather than represented as completed capabilities.